The Complete Practical Guide to Agile & Scrum for New Team Members

PART 1: THE FOUNDATIONS — WHY AGILE EXISTS

1.1 The Problem That Agile Solves

Before Agile existed, most software was built using the **Waterfall model** — a sequential
process where you complete one phase entirely before moving to the next:

```
Requirements → Design → Implementation → Testing → Deployment → Maintenance
```

This worked reasonably well for building bridges or aircraft — physical things where changing your mind halfway through construction is catastrophically expensive.
But software is fundamentally different.

The core problem with Waterfall for software:

- By the time software was delivered (often 1-3 years later), the business needs had changed

1.2 The Agile Manifesto — The Document That Changed Software Development

On February 11-13, 2001, seventeen software practitioners met at a ski resort in Snowbird,
Utah. They included Kent Beck (creator of XP), Ken Schwaber and Jeff Sutherland (co-creators of Scrum), Martin Fowler, and others.

They produced a document of just **68 words** that changed the industry:

```
We are uncovering better ways of developing software by doing it and
helping others do it. Through this work we have come to value:

Individuals and interactions   over   processes and tools
Working software               over   comprehensive documentation
Customer collaboration         over   contract negotiation
Responding to change           over   following a plan

That is, while there is value in the items on the right,
we value the items on the left more.
```

1.3 The 12 Agile Principles

Behind the manifesto sit 12 principles. Every Agile framework (Scrum, Kanban, SAFe, etc.)
traces back to these:

#	Principle
1	Our highest priority is to satisfy the customer through early and continuous delivery of valuable software
2	Welcome changing requirements, even late in development
3	Deliver working software frequently, from a couple of weeks to a couple of months
4	Business people and developers must work together daily throughout the project
5	Build projects around motivated individuals; give them the environment and trust they need

1.4 Agile Is a Philosophy, Not a Process

This is perhaps the most important thing to understand before joining any team:

Agile is NOT a process. It is a set of values and principles.

The processes that IMPLEMENT agile values include:

- **Scrum** — the most widely used framework
- **Kanban** — flow-based, no iterations
- **XP (Extreme Programming)** — engineering practices focused
- **SAFe (Scaled Agile Framework)** — for large organizations
- **LeSS (Large Scale Scrum)** — scaling Scrum
- **Scrumban** — hybrid of Scrum and Kanban

Most teams you will join use **Scrum** — sometimes pure Scrum, sometimes Scrum + some

PART 2: SCRUM — THE COMPLETE GUIDE

2.1 What Is Scrum?

Scrum is a **lightweight framework** for developing and sustaining complex products. It was created by Ken Schwaber and Jeff Sutherland in the early 1990s and has been continuously refined. The official definition comes from **The Scrum Guide** (scrumguides.org) — a free 13-page document that is the definitive reference.

Key word: **framework**, not methodology. Scrum gives you a structure — roles, events, artifacts, and rules. It does NOT tell you how to write code, how to design a database, or how to test software. Those are your team's choices.

Scrum is based on three pillars:

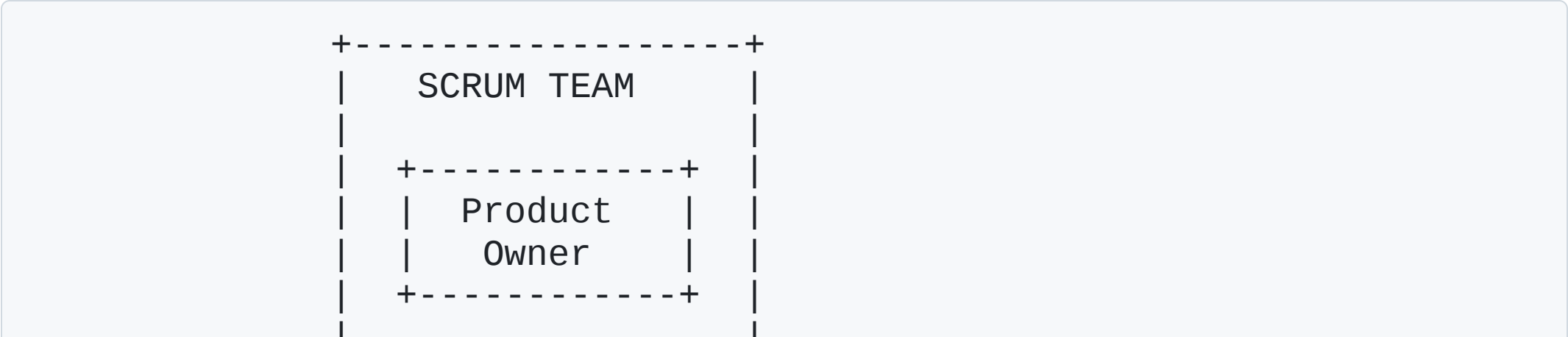
```
      TRANSPARENCY
     /
    /
```

2.2 The Scrum Team — Roles in Detail

A Scrum Team consists of **one Product Owner, one Scrum Master, and Developers**.

The team is:

- **Small** — typically 10 or fewer people (3-9 developers is optimal)
- **Cross-functional** — all skills needed to create value in each Sprint are present
- **Self-managing** — the team decides internally who does what, when, and how
- **No sub-teams, no hierarchies** — everyone is equal within the team



2.2.1 The Product Owner

Who they are: The person accountable for maximizing the value of the product resulting from the work of the Scrum Team.

Their primary responsibilities:

- Developing and explicitly communicating the **Product Goal**
- Creating and clearly communicating **Product Backlog items**
- **Ordering** (prioritizing) Product Backlog items
- Ensuring the Product Backlog is transparent, visible, and understood

Critical things to understand about the PO:

The Product Owner is **one person**, not a committee. Their decisions must be respected.
If someone wants to change the priority, they must convince the PO, not override them.

2.2.2 The Scrum Master

Who they are: Accountable for establishing Scrum as defined in the Scrum Guide.
They help everyone understand Scrum theory and practice.

Important: The Scrum Master is a **servant-leader**, not a manager.
They do not assign tasks. They do not have authority over developers.

Their responsibilities toward the Developers:

- Coaching the team in self-management and cross-functionality
- Helping focus on creating high-value Increments that meet the Definition of Done
- Causing removal of impediments (blockers) to the team's progress
- Ensuring all Scrum events take place and are positive and productive

What to go to your Scrum Master for:

- You are blocked and cannot make progress

2.2.3 The Developers

Who they are: The people in the Scrum Team committed to creating any aspect of a usable Increment each Sprint.

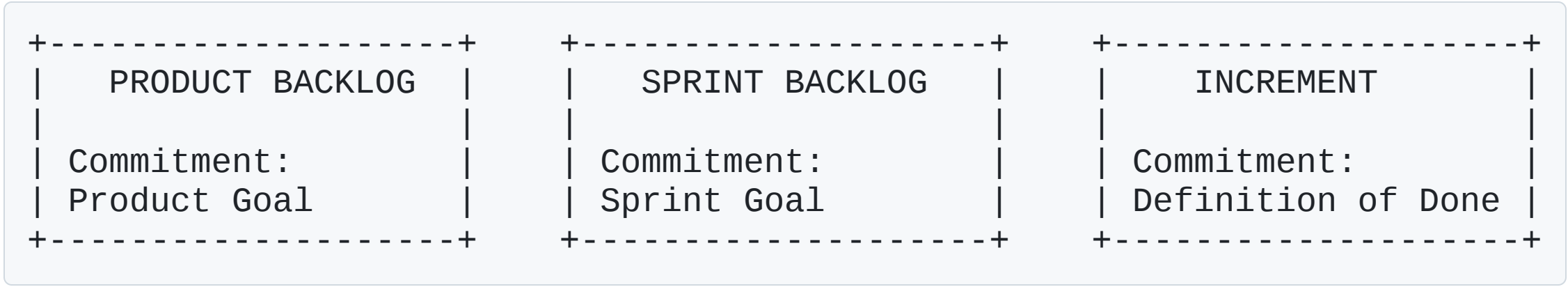
Key responsibilities:

- Creating a plan for the Sprint — the **Sprint Backlog**
- Instilling quality by adhering to a **Definition of Done**
- Adapting their plan each day toward the Sprint Goal
- Holding each other accountable as professionals

Important nuances:

- "Developer" doesn't just mean software developer — it means anyone who contributes:
  developers, testers, designers, DBAs, etc.
- There are NO titles within Developers in Scrum. No "senior developer" who

2.3 Scrum Artifacts — What You Track and Create



2.3.1 The Product Backlog

Definition: An emergent, ordered list of what is needed to improve the product.
It is the single source of work undertaken by the Scrum Team.

Key properties:

- **Never complete** — it evolves as the product and environment evolve
- **Ordered by value** — highest priority items at the top
- **Transparent** — visible to everyone
- **Maintained by the Product Owner**

What goes in the Product Backlog:

- User Stories
- Features
- Bug fixes

2.3.2 The Sprint Backlog

Definition: Composed of the Sprint Goal (why), the set of Product Backlog items selected for the Sprint (what), and an actionable plan for delivering the Increment (how).

Key properties:

- Owned by **Developers** — NOT the Product Owner
- A **real-time picture** of the work planned for the Sprint
- Updated throughout the Sprint as more is learned

The Sprint Goal:

- The single objective for the Sprint
- Provides flexibility while giving focus and coherence
- If the work turns out to be different than expected, developers collaborate with PO

2.3.3 The Increment

Definition: A concrete stepping stone toward the Product Goal. Each Increment is additive to all prior Increments and thoroughly verified, ensuring all Increments work together.

Key rule: An Increment must be **usable** in order to provide value. The PO may choose not to release it, but it must be in a state where it COULD be released.

Multiple Increments may be created in one Sprint.

2.3.4 Definition of Done (DoD)

This is one of the most practically important concepts for a new team member.

Definition: A formal description of the state of the Increment when it meets the quality measures required for the product.

Why it matters:

- Creates shared understanding of what "done" means
- Ensures transparency — everyone knows when work is complete
- Prevents the "90% done" syndrome
- Ensures quality is built in, not inspected in at the end

Example Definition of Done for a web application team:

```
Definition of Done - Example

Code:
  [ ] Code written and self-reviewed
```

2.4 Scrum Events — The Ceremonies

All Scrum events are formal opportunities to inspect and adapt. Each event has a fixed maximum duration (time-boxed).

```
SPRINT (1-4 weeks, fixed length)
|
+-- Sprint Planning (beginning of Sprint)
|
+-- Daily Scrum (every day)
|
+-- Sprint Review (end of Sprint)
|
+-- Sprint Retrospective (end of Sprint, after Review)
```

2.4.1 The Sprint

Definition: A fixed-length event (1-4 weeks, usually 2 weeks) during which a "Done", usable, potentially releasable product Increment is created.

Key rules about Sprints:

- Fixed length — never extended
- A new Sprint starts immediately after the conclusion of the previous Sprint
- No changes are made that would endanger the Sprint Goal
- Quality does not decrease
- Scope may be clarified and renegotiated with the PO

Can a Sprint be cancelled?
Yes — but ONLY by the Product Owner. This is extremely rare and happens only when the Sprint Goal becomes obsolete.

2.4.2 Sprint Planning

Time-box: Maximum 8 hours for a one-month Sprint
(proportionally less for shorter Sprints — typically 2-4 hours for 2-week Sprints)

Who attends: The entire Scrum Team

What happens:

```
Sprint Planning addresses three topics:

TOPIC 1: WHY is this Sprint valuable?
  - PO proposes how the product could increase its value
  - The whole Scrum Team collaborates to define the Sprint Goal

TOPIC 2: WHAT can be Done this Sprint?
  - Developers select items from the Product Backlog
  - Only developers can make this assessment (not the PO)
  - Based on past performance (velocity) and team capacity

TOPIC 3: HOW will the chosen work get done?
```

2.4.3 Daily Scrum

Time-box: Maximum 15 minutes

When: Same time, same place, every day of the Sprint

Who: Developers (Scrum Master and PO may attend but are not required)

Purpose: Inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary.

The classic three questions (one common format):

```
1. What did I do YESTERDAY that helped the team
   meet the Sprint Goal?

2. What will I do TODAY to help the team
   meet the Sprint Goal?

3. Do I see any impediment that prevents me or
   the team from meeting the Sprint Goal?
```

2.4.4 Sprint Review

Time-box: Maximum 4 hours for a one-month Sprint
(typically 1-2 hours for 2-week Sprints)

Who attends: Scrum Team + key stakeholders (invited by PO)

Purpose: Inspect the outcome of the Sprint and determine future adaptations.

What happens:

```
Sprint Review Agenda (typical):

1. PO explains what PBIs have been "Done" and what has not
2. Team DEMONSTRATES working software (not slides!)
3. Discussion of what went well and what problems were encountered
4. Stakeholders ask questions and give feedback
5. Discussion of the Product Backlog as it stands
6. PO discusses likely target dates based on current progress
7. Review of timeline, budget, potential capabilities, marketplace
```

2.4.5 Sprint Retrospective

Time-box: Maximum 3 hours for a one-month Sprint
(typically 45-90 minutes for 2-week Sprints)

Who attends: Scrum Team ONLY (no external stakeholders)

Purpose: Inspect how the last Sprint went regarding individuals, interactions, processes, tools, and Definition of Done. Identify improvements to be enacted in next Sprint.

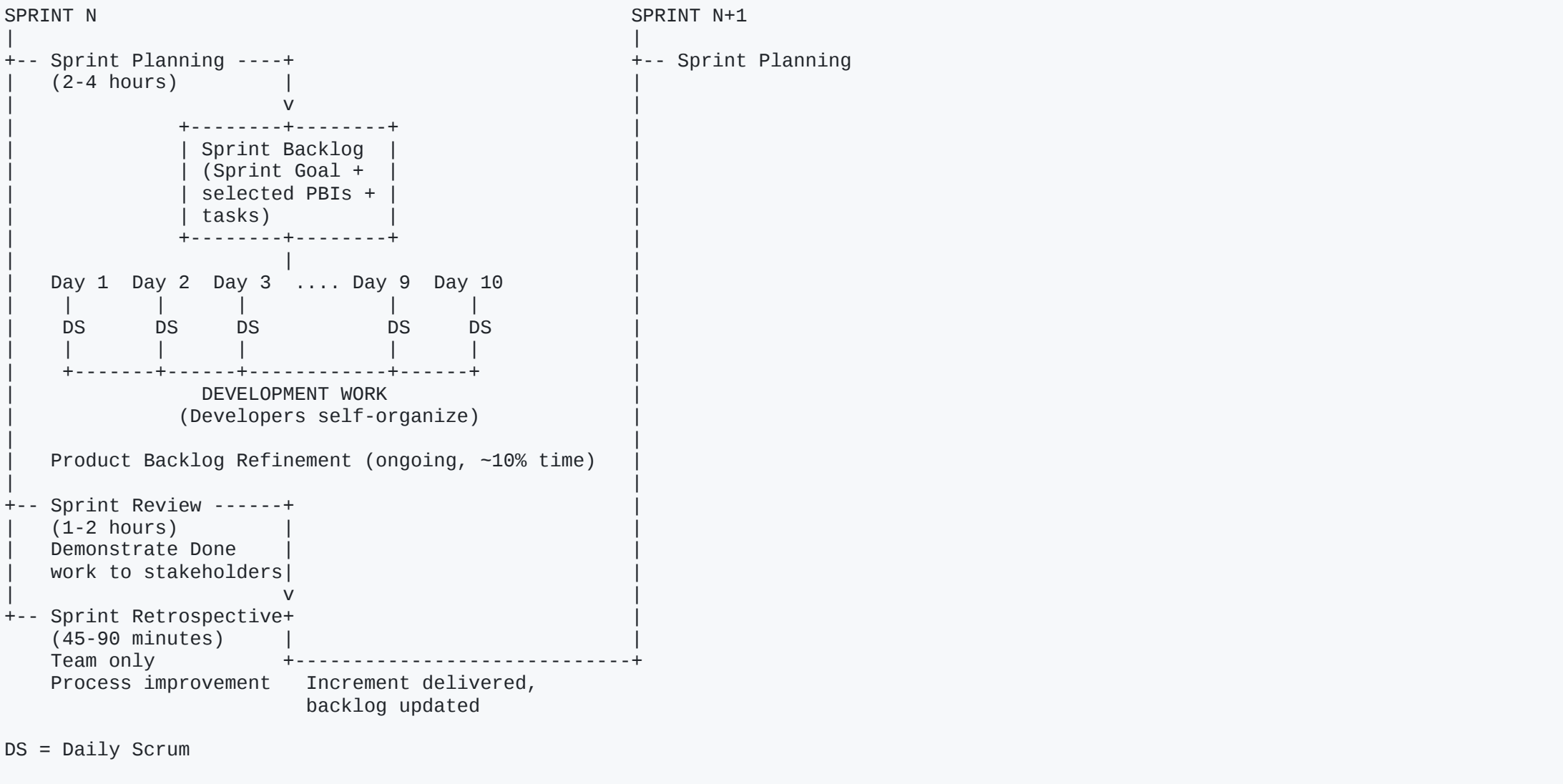
This is the most important ceremony for team health.

The Retrospective addresses three questions:

```
1. What went WELL during the last Sprint?
   (Don't skip this — celebrate success)

2. What could be IMPROVED?
   (The core of the retro)
```

2.5 The Sprint — The Full Picture



PART 3: USER STORIES IN DEPTH

3.1 What Is a User Story?

A User Story is a brief, simple description of a feature told from the perspective of the person who desires the new capability.

Standard format:

```
As a [type of user],
I want [some goal]
so that [some reason/benefit].
```

Examples:

```
As a registered customer,
I want to save my payment details,
so that I can check out faster on future purchases.

As an administrator,
```

3.2 The INVEST Criteria for Good User Stories

```
I - INDEPENDENT
    Stories should be self-contained and not dependent on other stories.
    This allows them to be prioritized and delivered in any order.

N - NEGOTIABLE
    Stories are NOT fixed contracts. Details are to be negotiated.
    The story is a starting point for conversation, not a spec.

V - VALUABLE
    Every story must deliver value to the user or business.
    "Refactor database layer" is not a user story — it's a technical task.

E - ESTIMABLE
    The team must be able to estimate the size.
    If it can't be estimated, it needs more discussion or splitting.

S - SMALL
    Stories should be small enough to be completed within one Sprint.
    Ideally, several stories per Sprint per developer.

T - TESTABLE
    There must be clear criteria for when the story is done.
    If you can't test it, it's too vague.
```

3.3 Acceptance Criteria

Acceptance Criteria define the conditions that a solution must meet to be accepted by the Product Owner. They answer "How do we know this story is done?"

Common format — Given/When/Then (Gherkin syntax):

```
SCENARIO: Successful password reset

Given: a registered user is on the login page
  And: the user has forgotten their password
When: the user clicks "Forgot Password" and enters their email address
Then: the user should receive a password reset email within 2 minutes
  And: the email should contain a unique reset link valid for 24 hours
  And: the user should see a confirmation message on screen

SCENARIO: Password reset with unregistered email

Given: a user is on the "Forgot Password" page
When: the user enters an email address not registered in the system
Then: the user should see a message "If this email is registered,
      you will receive a reset link"
```

3.4 Story Splitting — Breaking Large Stories Down

An Epic is a large body of work that must be broken into smaller stories:

```
EPIC: User can manage their account

STORIES:
  - As a user, I want to change my email address
  - As a user, I want to change my password
  - As a user, I want to update my profile picture
  - As a user, I want to delete my account
  - As a user, I want to see my login history
```

Common splitting patterns:

```
BY WORKFLOW STEPS:
  "User can complete checkout" ->
  - User can view cart
  - User can enter shipping details
  - User can enter payment details
  - User can confirm and place order
  - User can see order confirmation
```

PART 4: ESTIMATION IN SCRUM

4.1 Why We Estimate

Estimation in Scrum serves several purposes:

- Helps the team understand the complexity of work
- Allows the PO to make informed prioritization decisions
- Enables sprint planning — how much can we take on?
- Builds shared understanding of what needs to be done

Key insight: Estimation is not about precision. It is about **relative sizing** and **shared understanding**.

4.2 Story Points

Story Points are a unit of measure for expressing an estimate of the overall effort required to fully implement a Product Backlog item.

They capture a combination of:

- **Volume** — how much work is there?
- **Complexity** — how difficult is it?
- **Risk/Uncertainty** — how much is unknown?

Story Points are NOT hours. This is crucial. 5 story points does not mean 5 hours. Different developers will take different amounts of time to complete a 5-point story. That's fine — story points are a team measure, not individual.

4.3 The Fibonacci Scale

Most teams use a modified Fibonacci sequence for story points:

```
1 - 2 - 3 - 5 - 8 - 13 - 21 - 34 - 55 - 89

Or simplified:
1 - 2 - 3 - 5 - 8 - 13 - 20 - 40 - 100
```

Why Fibonacci and not 1-2-3-4-5?

Because the uncertainty of estimation grows as the size grows. The gap between 20 and 21 is meaningless, but the gap between 20 and 40 is significant. Fibonacci forces honest acknowledgment of uncertainty.

Special values:

- **0** — trivial, essentially no work
- **? or infinity** — too large or too uncertain; needs splitting/refinement

4.4 Planning Poker

Planning Poker is the most commonly used estimation technique in Scrum teams.

How it works:

```
STEP 1: Product Owner reads and explains a User Story

STEP 2: Developers ask clarifying questions until they
        understand the full scope

STEP 3: Each Developer silently picks a card with their estimate
        (physical cards or digital: PlanningPoker.com,
         Scrum Poker Online, or built into Jira)

STEP 4: SIMULTANEOUSLY, everyone reveals their cards
        (simultaneous reveal is critical — prevents anchoring)

STEP 5: If consensus (or close) — that's the estimate. Done.

STEP 6: If large disagreement — the HIGHEST and LOWEST estimators
        explain their reasoning
```

4.5 Velocity

Velocity is the amount of work a Scrum Team completes during a single Sprint, measured in story points.

```
Sprint 1:  Completed 23 story points
Sprint 2:  Completed 31 story points
Sprint 3:  Completed 27 story points
Sprint 4:  Completed 29 story points

Average velocity: (23+31+27+29)/4 = 27.5 story points per sprint
```

How velocity is used:

In Sprint Planning, the team uses historical velocity to determine how many story points they can commit to. If your velocity is ~28, don't commit to 50 points.

Important caveats about velocity:

```
NEVER use velocity to:
```

PART 5: THE SCRUM BOARD AND TRACKING TOOLS

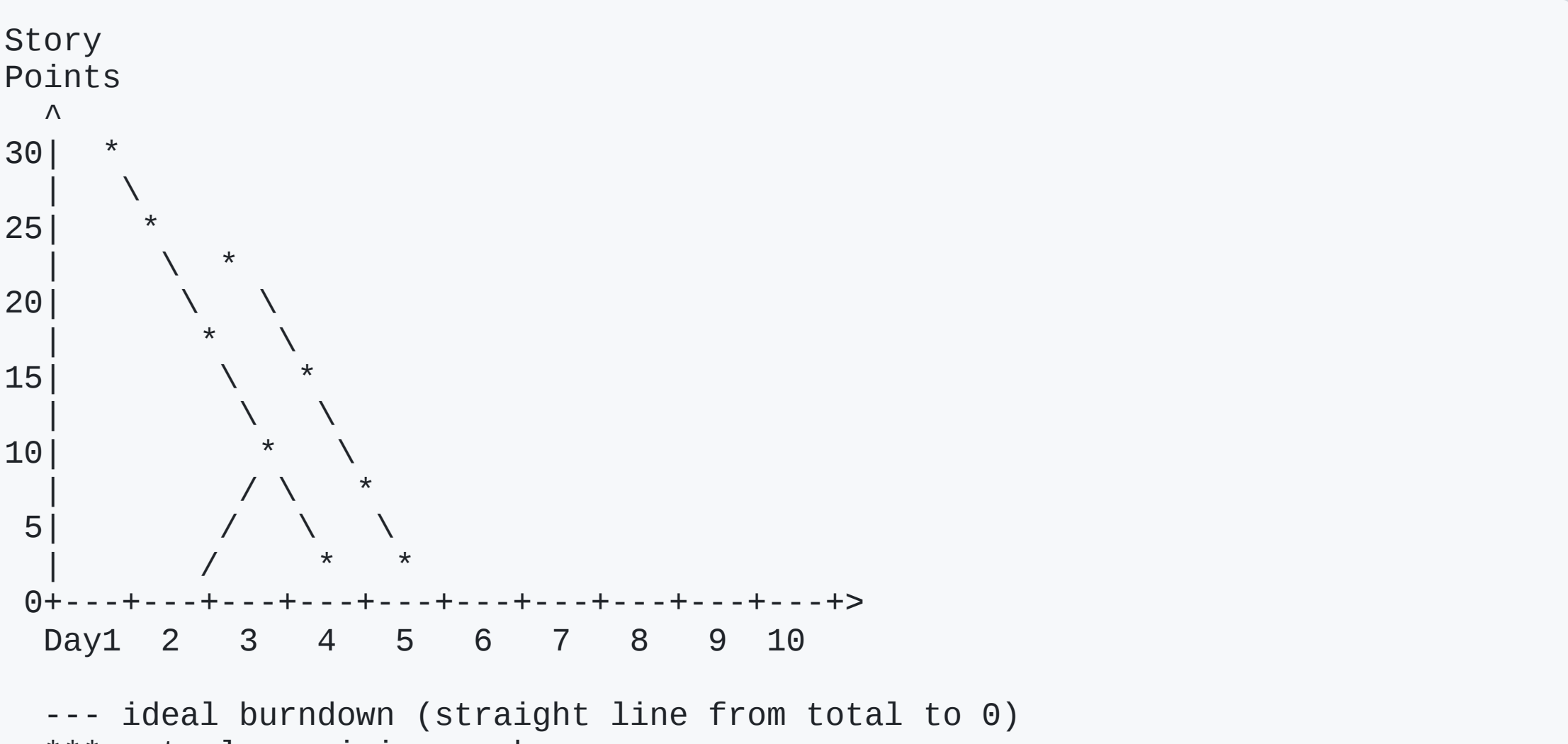
5.1 The Scrum Board / Kanban Board

Basic board layout:

```
+--------------+--------------+--------------+--------------+
|   TO DO      | IN PROGRESS  |  IN REVIEW   |    DONE      |
+--------------+--------------+--------------+--------------+
|              |              |              |              |
| [Story A]    | [Story C]    | [Story B]    | [Story Z]    |
|   5 pts      |   3 pts      |   8 pts      |   13 pts     |
|              |              |              |              |
| [Story D]    | [Task C-1]   |              | [Story Y]    |
|   2 pts      | @john        |              |   5 pts      |
|              |              |              |              |
| [Story E]    |              |              |              |
|   8 pts      |              |              |              |
+--------------+--------------+--------------+--------------+
Sprint Goal: "Enable customers to complete checkout end-to-end"
Sprint: 2024-01-15 to 2024-01-26  |  Velocity Target: 29pts
```

5.2 Burndown Charts

A **Sprint Burndown Chart** shows remaining work over the days of the Sprint.



5.3 Common Digital Tools

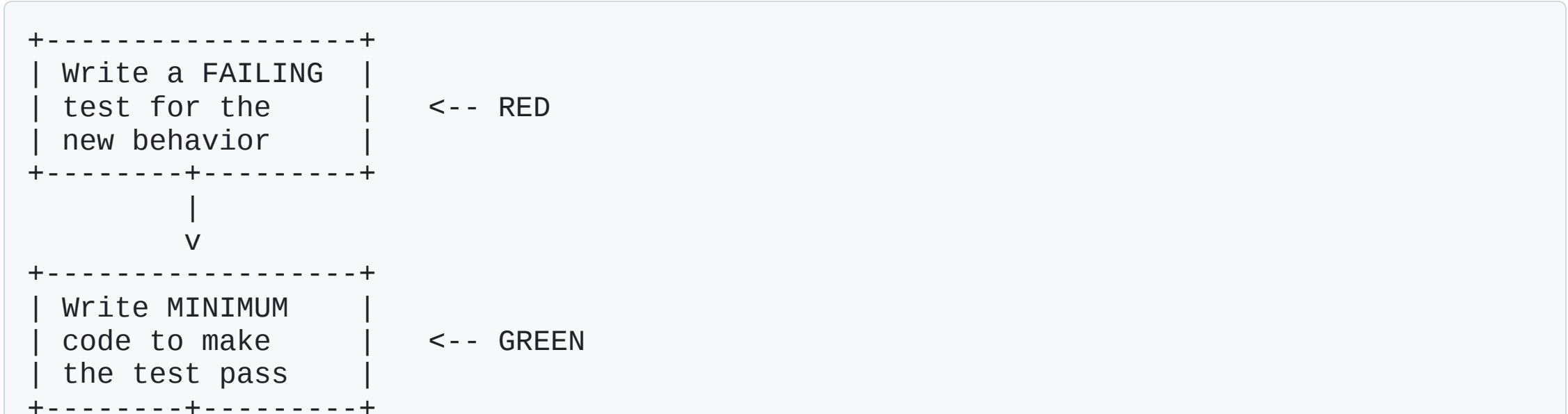
Tool	Used For	Notes
Jira	Full Scrum/Kanban management	Industry standard; complex but powerful
Azure DevOps	Boards + CI/CD integration	Preferred in Microsoft/.NET shops
Linear	Modern issue tracking	Popular in startups; clean UI
Trello	Simple Kanban boards	Good for small teams
GitHub Projects	Kanban for GitHub-heavy teams	Free, integrated with code
Confluence	Documentation/wiki	Often paired with Jira
Miro/Mural	Remote retrospectives,	Virtual whiteboards

PART 6: AGILE ENGINEERING PRACTICES

Scrum is a management framework — it says nothing about HOW to write code.
But most effective Agile teams combine Scrum with engineering practices,
often from **Extreme Programming (XP)**.

6.1 Test-Driven Development (TDD)

The Red-Green-Refactor cycle:



6.2 Continuous Integration (CI)

Definition: Developers integrate (merge) their code into a shared branch frequently — at least once per day. Each integration is verified by an automated build and automated tests.

```
Developer writes code
         |
         v
Developer runs tests locally — all pass
         |
         v
Developer pushes to feature branch
         |
         v
CI Pipeline triggers automatically:
  - Build the code
  - Run all unit tests
  - Run integration tests
  - Run static analysis / linting
  - Check code coverage
         |
    PASS?          FAIL?
```

6.3 Code Review / Pull Requests

Almost all Agile teams practice code review. Here's the typical workflow:

```
1. Developer creates a feature branch
   (e.g., feature/AUTH-42-password-reset)

2. Developer writes code and commits

3. Developer opens a Pull Request (PR) / Merge Request (MR)
   - Links to Jira ticket
   - Describes what was changed and why
   - Requests reviewers

4. Reviewer(s) read the code and leave comments:
   - Questions (seeking understanding)
   - Suggestions (optional improvements)
   - Required changes (must fix before merge)

5. Developer addresses feedback

6. Reviewer approves

7. Code merged to main branch
```

6.4 Refactoring

Definition: Restructuring existing code without changing its external behavior, to improve readability, maintainability, or extensibility.

The key principle: Never refactor AND add features at the same time.
One commit = one purpose.

Common refactoring operations:

- Rename variable/method/class to better describe its purpose
- Extract method (pull repeated code into a function)
- Extract class (when a class is doing too much)
- Move method to the class where it logically belongs
- Simplify conditionals
- Remove dead code

6.5 Pair Programming

Two developers work together at one computer.

```
DRIVER:    Focuses on the immediate task of writing code
NAVIGATOR: Thinks about the bigger picture, spots mistakes,
           suggests improvements

Roles switch frequently (e.g., every 25 minutes)
```

Benefits:

- Fewer bugs (two sets of eyes)
- Knowledge transfer (especially useful for onboarding)
- Better design decisions (discussed before coded)
- Reduces knowledge silos

Note: Not all teams practice pair programming.

It's an XP practice that some Scrum teams adopt.

PART 7: COMMON CHALLENGES AND HOW TO HANDLE THEM

7.1 When the Sprint Goes Wrong

Scenario: Mid-sprint you realize the Sprint Goal won't be met

```
DO:
  1. Raise it at the Daily Scrum immediately
  2. Discuss with the team: what can be done?
  3. Scrum Master may facilitate conversation with PO
  4. Negotiate scope reduction (remove lower-priority items)
     while protecting the Sprint Goal
  5. Never silently fail to deliver — transparency is a core value

DON'T:
  - Hide the problem hoping it fixes itself
  - Work overtime without team agreement
  - Add people to the Sprint
```

7.2 Handling Changing Requirements

```
New requirement appears:

DURING THE SPRINT:
  - PO receives the new requirement
  - If critical: negotiate with team to add it,
    replacing equivalent scope
  - Sprint Goal should not change
  - The Sprint Backlog can be adjusted with team agreement

AFTER THE SPRINT (normal case):
  - PO adds it to the Product Backlog
  - PO prioritizes it relative to everything else
  - It gets picked up in a future Sprint

Rule of thumb:
  - Urgent + high value   -> negotiate into current Sprint
  - Not urgent            -> Product Backlog -> next Sprint
  - Urgent + low value    -> PO decides
```

7.3 Dealing with Technical Debt

Technical debt is the accumulated cost of shortcuts, poor design decisions, and deferred refactoring that makes the codebase increasingly difficult to work with.

```
TECHNICAL DEBT ANALOGY:
  Taking on debt = makes you faster now, costs more later

  If ignored long enough:
  New features take longer -> more bugs -> slower velocity
  -> more pressure -> more shortcuts -> more debt -> death spiral
```

How Scrum teams manage technical debt:

- Allocate a percentage of each Sprint to technical debt reduction (typically 10-20%)
- Add technical debt items to the Product Backlog
- Include refactoring in the Definition of Done
- Make debt visible to the PO — it affects velocity and is a business concern

7.4 The Definition of Ready

Some teams define a "Definition of Ready" — criteria that a backlog item must meet before it can be picked up in Sprint Planning.

```
Definition of Ready - Example:
  [ ] User story clearly written (As a... I want... So that...)
  [ ] Acceptance Criteria defined and agreed with PO
  [ ] Story estimated by the team
  [ ] Dependencies identified
  [ ] No unresolved questions that would block development
  [ ] UI mockup available (for front-end stories)
  [ ] Story fits within one Sprint

If a story doesn't meet DoR -> it should NOT go into Sprint Planning
```

7.5 Scrum Anti-Patterns — Red Flags

You may join a team that claims to do Scrum but practices "ScrumBut":

"We use Scrum, but..."

```
ANTI-PATTERN: "We do Scrum but we don't do Retrospectives"
PROBLEM: No mechanism for process improvement

ANTI-PATTERN: "We do Scrum but the PO is never available"
PROBLEM: Team can't get clarification; stories delivered wrong

ANTI-PATTERN: "Management changes our Sprint mid-way"
PROBLEM: Team can't focus; commitment is meaningless

ANTI-PATTERN: "Daily Scrum is a 1-hour status meeting"
PROBLEM: Waste of time; becomes demoralizing

ANTI-PATTERN: "We never actually ship anything"
PROBLEM: Increment not potentially shippable; waterfall in disguise

ANTI-PATTERN: "We estimate in hours, not story points"
PROBLEM: Individual performance pressure

ANTI-PATTERN: "The Scrum Master assigns tasks to developers"
PROBLEM: SM acting as manager, killing self-organization
```

PART 8: SCRUM OF SCRUMS AND SCALING

When multiple Scrum Teams work on the same product:

```
SINGLE TEAM SCRUM:
  Product Owner -> Product Backlog -> Scrum Team -> Increment

MULTI-TEAM SCRUM (Scrum of Scrums):
  Product Owner
       |
  Product Backlog
       |
  +----+----+----+
  |    |    |    |
  Team Team Team Team
  A    B    C    D
  |    |    |    |
  +----+----+----+
       |
  Daily "Scrum of Scrums":
  One representative per team meets daily
  Same three questions, but at team level
       |
```

PART 9: PRACTICAL ONBOARDING GUIDE

9.1 Your First Day

```
Checklist - Day 1:
  [ ] Get access to Jira/Azure DevOps/whatever tool the team uses
  [ ] Get added to team Slack/Teams channels
  [ ] Get access to the code repository (GitHub/GitLab/Bitbucket)
  [ ] Get development environment setup instructions (README)
  [ ] Read the team's Definition of Done
  [ ] Read the team's Working Agreement (if it exists)
  [ ] Find out when the next Daily Scrum is
  [ ] Find out which Sprint the team is in and what day
  [ ] Ask to see the current Sprint Backlog
  [ ] Ask to see the Product Backlog
  [ ] Find out who is the Product Owner and Scrum Master
```

9.2 Your First Week

```
Checklist - Week 1:
  [ ] Attend at least 3 Daily Scrums (observe before speaking actively)
  [ ] Set up your development environment and run the project locally
  [ ] Read the top 10 items in the Product Backlog
  [ ] Ask your Scrum Master or buddy for a codebase walkthrough
  [ ] Read the team's README, CONTRIBUTING guide, and any wiki
  [ ] Attend the Sprint Review (if one happens this week)
  [ ] Attend the Retrospective (if one happens)
  [ ] Identify one small task you can complete this week
  [ ] Learn the team's Git branching strategy (GitFlow? trunk-based?)
  [ ] Learn the team's PR/code review process
```

9.3 Your First Sprint

```
Sprint 1 goals for a new team member:
  - Complete 2-3 small, well-defined stories
  - Participate in Sprint Planning actively
  - Raise blockers on Daily Scrum the moment they appear
  - Have at least one PR approved and merged
  - Contribute to Retrospective
  - Don't over-commit — under-promise, over-deliver
```

9.4 Questions to Ask

At Sprint Planning:

- "Can you clarify what 'done' looks like for this story?"
- "What are the acceptance criteria for this story?"
- "Is there existing code I should look at before starting this?"
- "Are there dependencies I should know about?"

At Daily Scrum:

- "I'm blocked on X — is there someone who can help me after this?"
- "I finished Y — should I pick up Z or is there something higher priority?"

At Retrospective:

- "I noticed that... — is this something we usually do?"
- "As someone new, I found it hard to... — is there a better way?"

PART 10: CERTIFICATIONS AND FURTHER LEARNING

10.1 Recommended Learning Path

```
LEVEL 1 - Before joining the team (do this NOW):
  1. Read The Scrum Guide (scrumguides.org) - FREE, 13 pages, 30 minutes
  2. Read this guide
  3. Watch: "Intro to Scrum in Under 10 Minutes" (YouTube, Axosoft)

LEVEL 2 - First month on the job:
  4. Essential Scrum - Kenneth Rubin (book)
  5. User Stories Applied - Mike Cohn (book)
  6. Attend a free webinar on Scrum.org

LEVEL 3 - After 3-6 months of practice:
  7. Agile Estimating and Planning - Mike Cohn (book)
  8. Clean Code - Robert C. Martin (book) - engineering practices
  9. The Phoenix Project - Kim, Behr, Spafford (novel, DevOps culture)

LEVEL 4 - Certification (optional but valuable):
  10. PSM I - Professional Scrum Master (Scrum.org) ~$150
```

10.2 Certifications Overview

Certification	Provider	Cost	Difficulty	Best For
PSM I	Scrum.org	~$150	Medium	Developers & SM wannabes
PSM II	Scrum.org	~$250	Hard	Experienced SMs
PSPO I	Scrum.org	~$200	Medium	PO wannabes
CSM	ScrumAlliance	~$400+	Easy (needs course)	Broad recognition
PMI-ACP	PMI	~$495	Hard	Broad Agile knowledge
SAFe Agilist	Scaled Agile	~$995	Medium	Large enterprise

Recommendation for a student/junior developer:

PART 11: GLOSSARY — QUICK REFERENCE

Term	Definition
Acceptance Criteria	Conditions a story must meet to be accepted by PO
Backlog Refinement	Ongoing process of detailing and estimating backlog items
Burndown Chart	Graph showing remaining work vs. time in Sprint
Daily Scrum	15-minute daily synchronization for Developers
Definition of Done	Checklist of quality criteria for a completed Increment
Definition of Ready	Criteria a PBI must meet before Sprint Planning
Epic	Large user story that must be broken down
Impediment	Anything blocking progress; removed by Scrum Master
Increment	Potentially shippable result of a Sprint

THE 10 MOST IMPORTANT THINGS TO REMEMBER

1. SCRUM IS A FRAMEWORK, NOT A PROCESS
   It gives you structure. You fill in the how.

2. TRANSPARENCY IS EVERYTHING
   If you're blocked - say it. If you're behind - say it.
   The worst thing you can do is hide problems.

3. THE SPRINT GOAL IS YOUR NORTH STAR
   Every decision in the Sprint should serve the Sprint Goal.

4. DONE MEANS DONE
   If it doesn't meet the Definition of Done, it is NOT done.
   "Almost done" is not done.

5. STORY POINTS ARE NOT HOURS
   They measure relative complexity. Don't stress about
   "how many hours is a 5?"

6. VELOCITY IS A TEAM METRIC
   Never used to compare individuals or teams.

7. THE SCRUM MASTER IS NOT YOUR MANAGER
   They remove obstacles. You manage your own work.

8. THE DAILY SCRUM IS NOT A STATUS REPORT
   It's for the TEAM to synchronize. Speak to the Sprint Goal.

9. RETROSPECTIVES ARE SACRED
   Be honest. Be constructive. The team that improves together
   ships better software together.

10. THE SCRUM GUIDE IS 13 PAGES
    Read it. All of it. It's free. Do it before your first day.
    scrumguides.org